JAVA — M8 SPRING WEBSERVICES

Answer Key — Spring Boot REST, JPA, Security, HttpClient & AI Assessment

SECTION A — Conceptual Understanding

Q1. Explain the annotations for /menu and /menu/{id}.

@RestController marks the class as a REST controller. @RequestMapping("/menu") defines the base path, @GetMapping handles GET requests, and @PathVariable binds the {id} value from the URL to a Java method parameter. @RestController differs from @Controller because its handler methods return response data directly, normally serialized as JSON, rather than resolving a view.

```
@RestController
@RequestMapping("/menu")
public class MenuController {
    @GetMapping
    public List<MenuItem> getMenu() { return menuItems; }

    @GetMapping("/{id}")
    public ResponseEntity<MenuItem> getById(@PathVariable Long id) {
        return menuItems.stream()
            .filter(x -> x.getId().equals(id))
            .findFirst()
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}
```

Logic:

- @GetMapping maps an HTTP GET request to a method.
- @PathVariable extracts a URI template value such as /menu/7 -> id=7.
- Returning ResponseEntity.notFound() produces HTTP 404 when the dish does not exist.

Q2. Explain CrudRepository and the JPA annotations on Order.

CrudRepository supplies standard persistence operations such as save(), findAll(), findById(), and deleteById(), avoiding repetitive JDBC CRUD SQL. The Order class is mapped as a JPA entity with @Entity, @Table, @Id, @GeneratedValue, and @Column. @GeneratedValue delegates primary-key generation to the selected strategy.

```
@Entity
@Table(name = "orders")
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable=false) private String customerName;
    @Column(nullable=false) private String itemName;
    @Column(nullable=false) private int quantity;
    @Column(nullable=false) private double totalAmount;
}

public interface OrderRepository
        extends CrudRepository<Order, Long> {
    Iterable<Order> findByCustomerName(String name);
}
```

Logic:

- @Entity makes Order persistent.
- @Id identifies the primary key.
- @GeneratedValue lets the persistence/database strategy generate IDs.
- findByCustomerName is a derived query method; no implementation is required.

Q3. Explain Basic Authentication and validation with @Valid.

Spring Security Basic Authentication uses the HTTP Authorization header with the Basic scheme. Spring Security authenticates the supplied credentials before allowing protected requests. For validation, @Valid on the request body triggers Bean Validation; @NotBlank rejects missing/blank strings and @Min(1) requires a numeric value of at least 1.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    SecurityFilterChain security(HttpSecurity http) throws Exception {
        return http
          .csrf(csrf -> csrf.disable())
          .authorizeHttpRequests(a -> a
             .requestMatchers("/admin/**").authenticated()
             .anyRequest().permitAll())
          .httpBasic(Customizer.withDefaults())
          .build();
    }
}
```

```
public class OrderRequest {
    @NotBlank private String customerName;
    @NotBlank private String itemName;
    @Min(1) private int quantity;
}

@PostMapping("/orders")
public ResponseEntity<?> place(@Valid @RequestBody OrderRequest req) {
    return ResponseEntity.ok(req);
}
```

Logic:

- Validation occurs before the normal service operation when the request fails its constraints.
- A validation exception handler can convert violations into HTTP 400 with field-level messages.
- Basic Authentication protects /admin/** according to the security rule.

Q4. Explain HttpClient GET and Jackson deserialization.

HttpClient builds and sends the GET request. The response body is read as a String and Jackson ObjectMapper.readValue() converts the JSON into a POJO. Normally the POJO property names correspond to the JSON keys; @JsonProperty explicitly maps a Java property when the JSON naming differs.

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://jsonplaceholder.typicode.com/todos/1"))
    .GET().build();

HttpResponse<String> response = client.send(
    request, HttpResponse.BodyHandlers.ofString());

ObjectMapper mapper = new ObjectMapper();
Todo todo = mapper.readValue(response.body(), Todo.class);

public class Todo {
    public int userId;
    public int id;
    public String title;
    public boolean completed;
}
```

Logic:

- HttpClient handles HTTP transport.
- ObjectMapper handles JSON/object mapping.
- @JsonProperty("delivery_time_minutes") can map a JSON snake_case key to a Java field named deliveryTimeMinutes.

Q5. Explain OpenAI Chat Completions integration.

The /recommend endpoint accepts a dietary preference, builds a JSON body containing model and messages, and sends an HTTP POST to the specified Chat Completions endpoint. The request uses Content-Type: application/json and Authorization: Bearer . The key should come from environment/external configuration rather than source code.

```
POST https://api.openai.com/v1/chat/completions
Content-Type: application/json
Authorization: Bearer <API_KEY>

{
  "model": "MODEL_NAME",
  "messages": [
    {"role":"user","content":"Suggest a meal for a vegetarian user."}
  ]
}
```

Logic:

- The assessment specifies POST and application/json.
- The response is parsed with Jackson and the assistant text is read from choices[0].message.content.
- Store the API key in an environment variable or external configuration.

Q6. Explain prompt templates and chaining in LangChain4J.

A prompt template is a reusable instruction pattern with placeholders for changing values such as a customer question or order status. A plain string is a single already-formed prompt. Chaining multiple prompt/model steps lets the output or context of one step feed a later step, allowing a chatbot to combine conversation context, order information, and consistent response instructions instead of making one isolated call.

Logic:

- Templates separate reusable instructions from variable data.
- Chaining supports multi-step, context-aware responses.
- A consistent instruction can preserve the required brand-appropriate tone.

SECTION B — Practical Coding Tasks

Task 1: Food Menu REST Controller

```
@RestController
@RequestMapping("/menu")
public class MenuController {
    private final List<MenuItem> menu = new ArrayList<>(List.of(
        new MenuItem(1L,"Paneer Biryani",280),
        new MenuItem(2L,"Margherita Pizza",420),
        new MenuItem(3L,"Masala Dosa",150)));

    @GetMapping
    public List<MenuItem> getMenu() { return menu; }

    @GetMapping("/{id}")
    public ResponseEntity<MenuItem> getById(@PathVariable Long id) {
        return menu.stream().filter(x -> x.getId().equals(id))
            .findFirst().map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<MenuItem> add(@RequestBody MenuItem item) {
        menu.add(item);
        return ResponseEntity.status(HttpStatus.CREATED).body(item);
    }
}

public class MenuItem {
    private Long id; private String name; private double price;
    public MenuItem() {}
    public MenuItem(Long id,String name,double price) {
        this.id=id; this.name=name; this.price=price;
    }
    // getters and setters
}
```

Logic:

- The list contains at least three MenuItem objects.
- GET /menu/{id} returns 404 when no ID matches.
- POST uses @RequestBody and returns HTTP 201.

Task 2: Order Persistence with Spring Data JPA

```
@Entity
@Table(name="orders")
public class Order {
    @Id @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;
    @Column(nullable=false) private String customerName;
    @Column(nullable=false) private String itemName;
    @Column(nullable=false) private int quantity;
    @Column(nullable=false) private double totalAmount;
    public Order() {}
    public Order(String c,String i,int q,double t) {
        customerName=c; itemName=i; quantity=q; totalAmount=t;
    }
    // getters and setters
}

public interface OrderRepository
        extends CrudRepository<Order,Long> {
    Iterable<Order> findByCustomerName(String name);
}

@Bean
CommandLineRunner runner(OrderRepository repo) {
    return args -> {
        repo.save(new Order("Riya Sharma","Paneer Biryani",2,560));
        repo.save(new Order("Aman Patel","Pizza",1,420));
        repo.findAll().forEach(System.out::println);
        repo.findByCustomerName("Riya Sharma")
            .forEach(System.out::println);
    };
}
```

Logic:

- save() persists the two startup examples.
- findAll() displays all records.
- findByCustomerName() demonstrates a derived repository query.

Task 3: Delivery Status Fetcher using HttpClient and Jackson

```
import com.fasterxml.jackson.databind.ObjectMapper;
import java.net.URI;
import java.net.http.*;

public class TodoFetcher {
```

```
    public static class Todo {
        public int userId;
        public int id;
        public String title;
        public boolean completed;
    }

    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();
        HttpRequest req = HttpRequest.newBuilder()
            .uri(URI.create(
              "https://jsonplaceholder.typicode.com/todos/1"))
            .GET().build();

        HttpResponse<String> res = client.send(
            req, HttpResponse.BodyHandlers.ofString());

        Todo todo = new ObjectMapper()
            .readValue(res.body(), Todo.class);

        System.out.println("User ID: " + todo.userId);
        System.out.println("ID: " + todo.id);
        System.out.println("Title: " + todo.title);
        System.out.println("Completed: " + todo.completed);
    }
}
```

Logic:

- The URL and GET method match the assessment.
- Jackson maps userId, id, title, and completed into Todo.
- All four fields are printed with clear labels.

Task 4: AI Meal Recommendation Endpoint

```
public record RecommendationRequest(String preference) {}

@RestController
public class RecommendationController {
    private final HttpClient client = HttpClient.newHttpClient();
    private final ObjectMapper mapper = new ObjectMapper();

    @Value("${openai.api-key}")
    private String apiKey;

    @PostMapping("/recommend")
    public ResponseEntity<?> recommend(
        @RequestBody RecommendationRequest request) {
        try {
            String body = mapper.writeValueAsString(Map.of(
              "model","MODEL_NAME",
              "messages",List.of(Map.of(
                "role","user",
                "content","Suggest a meal for: "
                           + request.preference()))));

            HttpRequest http = HttpRequest.newBuilder()
              .uri(URI.create(
                "https://api.openai.com/v1/chat/completions"))
              .header("Content-Type","application/json")
              .header("Authorization","Bearer "+apiKey)
              .POST(HttpRequest.BodyPublishers.ofString(body))
              .build();

            HttpResponse<String> r = client.send(
              http,HttpResponse.BodyHandlers.ofString());

            if (r.statusCode() < 200 || r.statusCode() >= 300)
              return ResponseEntity.status(502).body(
                Map.of("error","Recommendation API failed"));

            JsonNode root = mapper.readTree(r.body());
            String content = root.path("choices").get(0)
              .path("message").path("content").asText();

            return ResponseEntity.ok(
              Map.of("recommendation",content));
        } catch (Exception e) {
            return ResponseEntity.status(502).body(
              Map.of("error","Unable to obtain recommendation"));
        }
    }
}
```

```
# application.properties
openai.api-key=${OPENAI_API_KEY}
```

Logic:

- The endpoint accepts preference in JSON.
- The API response is read from choices[0].message.content.

- API failures are converted to HTTP 502.
- The key is supplied externally, not hardcoded.

SECTION C — Mini Capstone Project

Mini Project: Smart Food Delivery Backend with AI Recommendations

This solution combines the required REST endpoints, Spring Data JPA persistence, validation, Basic Authentication, and the OpenAI Chat Completions integration specified by the assessment.

Maven dependencies

```
spring-boot-starter-web
spring-boot-starter-data-jpa
spring-boot-starter-security
spring-boot-starter-validation
h2
jackson-databind
```

Order entity and repository

```
@Entity
@Table(name="orders")
public class Order {
    @Id @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;
    @Column(nullable=false) private String customerName;
    @Column(nullable=false) private String itemName;
    @Column(nullable=false) private int quantity;
    @Column(nullable=false) private double totalAmount;
    // constructors, getters, setters
}

public interface OrderRepository
        extends CrudRepository<Order,Long> {}
```

Validated POST /orders and GET /orders/{id}

```
public class OrderRequest {
    @NotBlank private String customerName;
    @NotBlank private String itemName;
    @Min(1) private int quantity;
    // getters and setters
}

@RestController
public class OrderController {
    private final OrderRepository repo;
    public OrderController(OrderRepository repo){this.repo=repo;}

    @PostMapping("/orders")
    public ResponseEntity<Order> place(
      @Valid @RequestBody OrderRequest r) {
        Order o = new Order(r.getCustomerName(),r.getItemName(),
            r.getQuantity(),r.getQuantity()*100.0);
        return ResponseEntity.status(201).body(repo.save(o));
    }

    @GetMapping("/orders/{id}")
    public ResponseEntity<Order> get(@PathVariable Long id) {
        return repo.findById(id).map(ResponseEntity::ok)
          .orElse(ResponseEntity.notFound().build());
    }
}
```

Admin security and endpoints

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    SecurityFilterChain filter(HttpSecurity http) throws Exception {
        return http.csrf(c -> c.disable())
          .authorizeHttpRequests(a -> a
            .requestMatchers("/menu","/orders","/recommend")
            .permitAll()
            .requestMatchers("/admin/**").authenticated()
            .anyRequest().authenticated())
          .httpBasic(Customizer.withDefaults())
          .build();
    }

    @Bean
    UserDetailsService users() {
        UserDetails admin = User.withUsername("admin")
          .password("{noop}admin123").roles("ADMIN").build();
        return new InMemoryUserDetailsManager(admin);
    }
}

@RestController
@RequestMapping("/admin")
class AdminController {
    private final OrderRepository repo;
    AdminController(OrderRepository repo){this.repo=repo;}
```

```
    @GetMapping("/orders")
    Iterable<Order> all(){ return repo.findAll(); }

    @DeleteMapping("/orders/{id}")
    ResponseEntity<Void> delete(@PathVariable Long id) {
        if(!repo.existsById(id)) return ResponseEntity.notFound().build();
        repo.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}
```

application.properties

```
spring.datasource.url=jdbc:h2:mem:quickeats
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
openai.api-key=${OPENAI_API_KEY}
```

Structured validation errors

```
@RestControllerAdvice
class ValidationHandler {
  @ExceptionHandler(MethodArgumentNotValidException.class)
  ResponseEntity<?> handle(MethodArgumentNotValidException ex) {
    Map<String,String> fields = new LinkedHashMap<>();
    ex.getBindingResult().getFieldErrors().forEach(e ->
      fields.put(e.getField(), e.getDefaultMessage()));
    return ResponseEntity.badRequest().body(Map.of(
      "status",400, "message","Validation failed",
      "fields",fields));
  }
}
```

Logic:

- The project exposes GET /menu, POST /orders, GET /orders/{id}, and POST /recommend.
- Orders are persisted through Spring Data JPA and OrderRepository.
- The /admin endpoints require Basic Authentication; unauthenticated requests are rejected.
- @Valid, @NotBlank, and @Min enforce the requested validation rules and the advice class returns structured HTTP 400 errors.
- The recommendation endpoint follows the required HttpClient/Jackson pattern.

SECTION D — AI-Augmented Learning

Step 1: Exact AI prompt

```
Write a Java console program that reads prompts from the user,
sends them to the OpenAI Chat Completions API using Java HttpClient,
uses Authorization: Bearer and a JSON model/messages body,
parses the response with Jackson, prints only
choices[0].message.content, and repeats until the user types exit.
Read the API key from OPENAI_API_KEY and return complete Java code.
```

AI's original code

```
String apiKey = System.getenv("OPENAI_API_KEY");
Scanner scanner = new Scanner(System.in);
HttpClient client = HttpClient.newHttpClient();
ObjectMapper mapper = new ObjectMapper();

while (true) {
    System.out.print("Prompt: ");
    String prompt = scanner.nextLine();
    if (prompt.equalsIgnoreCase("exit")) break;

    String json = mapper.writeValueAsString(Map.of(
      "model","MODEL_NAME",
      "messages",List.of(
        Map.of("role","user","content",prompt))));

    HttpRequest request = HttpRequest.newBuilder()
      .uri(URI.create("https://api.openai.com/v1/chat/completions"))
      .header("Content-Type","application/json")
      .header("Authorization","Bearer "+apiKey)
      .POST(HttpRequest.BodyPublishers.ofString(json)).build();

    HttpResponse<String> response = client.send(
      request,HttpResponse.BodyHandlers.ofString());

    JsonNode root = mapper.readTree(response.body());
    System.out.println(root.path("choices").get(0)
      .path("message").path("content").asText());
}
```

Corrected version

```
String apiKey = System.getenv("OPENAI_API_KEY");
if (apiKey == null || apiKey.isBlank()) {
    System.err.println("OPENAI_API_KEY is not set.");
    return;
}

try {
    HttpResponse<String> response = client.send(
        request, HttpResponse.BodyHandlers.ofString());

    if (response.statusCode() < 200 ||
        response.statusCode() >= 300) {
        System.err.println("API failed: HTTP "
            + response.statusCode());
        continue;
    }

    JsonNode root = mapper.readTree(response.body());
    JsonNode choices = root.path("choices");
    if (!choices.isArray() || choices.isEmpty()) {
        System.err.println("No assistant reply in response.");
        continue;
    }

    String reply = choices.get(0).path("message")
        .path("content").asText(null);

    if (reply == null || reply.isBlank()) {
        System.err.println("Assistant reply is missing.");
        continue;
    }

    System.out.println(reply);
} catch (Exception e) {
    System.err.println("Request error: " + e.getMessage());
}
```

3–4 line test/debug note

The original solution assumes the API key exists and that every response contains choices[0].message.content. I added an explicit API-key check, HTTP status handling, and response-shape validation so an API error or unexpected JSON does not terminate the loop unexpectedly. I also handle request exceptions and missing assistant content. The correction preserves the required repeated prompt-until-exit behavior.

Submission note: The assessment requires screenshots of program output or Postman responses. Capture those from your own execution; this answer key provides the source and expected structure.